

=====

1. PURPOSE

This document explains the database table structure, Redis session structure, data movement across tables, and business logic used in the AI customer support chatbot project.

The project handles an e-commerce order complaint flow:

1. User starts a complaint.
2. Bot asks for order ID.
3. Backend validates the order.
4. Backend creates a ticket.
5. User selects order item(s).
6. Backend stores selected ticket items.
7. User provides the issue type.
8. Backend checks duplicates.
9. Backend calculates refund.
10. Backend escalates or resolves the ticket.

The application is intentionally monolithic and enterprise-style:

- WebSocket controller only receives and returns messages.
- ChatService owns the conversation state machine.
- Domain services handle focused operations.
- PostgreSQL stores permanent data.
- Redis stores temporary conversation session state.
- AI gives structured signals only; backend makes final decisions.

2. MAIN STORAGE TYPES

PostgreSQL

PostgreSQL stores permanent data:

- Orders
- Order items
- Tickets
- Ticket items
- Chat messages

These records should remain even after Redis expires or the application restarts.

Redis

Redis stores temporary conversation state:

- Current user state
- Current ticket ID
- Selected item IDs
- Retry count

Redis key format:

```
'''text
session:{userId}
Page 1
```

'''

Example:

```
'''text
session:user-1
'''
```

TTL:

```
'''text
30 minutes
'''
```

Redis is not the source of truth for tickets. It only helps continue the live chat flow quickly.

3. POSTGRESQL TABLE STRUCTURE

The table names come from the JPA entity annotations in the code.

3.1 CUSTOMER_ORDERS

Entity:

```
'''text
CustomerOrder
'''
```

Java file:

```
'''text
src/main/java/com/personal/ai_chatbot/entity/CustomerOrder.java
'''
```

Purpose:

Stores the customer's order header.

Table:

```
'''text
customer_orders
'''
```

Columns:

Column	Java Type	DB Meaning	Notes
order_id	String	Primary key	Example: ORD-1001
user_id	String	Owner of the order	Used to validate that the order belongs to the current
total_amount	BigDecimal	Full order amount	Useful for reporting; refund uses selected ite

Relationship:

```
'''text
customer_orders 1 ---> many order_items
'''
```

Code mapping:

```
'''java
@OneToMany(mappedBy = "order", cascade = CascadeType.ALL, orphanRemoval = true)
private List<OrderItem> items = new ArrayList<>();
'''
```

Sample seeded row:

```
'''text
order_id: ORD-1001
user_id: user-1
total_amount: 8000
'''
```

3.2 ORDER_ITEMS

Entity:

```
'''text
OrderItem
'''
```

Java file:

```
'''text
src/main/java/com/personal/ai_chatbot/entity/OrderItem.java
'''
```

Purpose:

Stores real order items purchased by the customer.

Table:

```
'''text
order_items
'''
```

Columns:

Column	Java Type	DB Meaning	Notes
item_id	String	Primary key	Example: ITEM-1
order_id	String	Foreign key to customer_orders.order_id	Many items belong to one order
name	String	Product name	Example: Shoes
price	BigDecimal	Item price	Used to calculate refund

Relationship:

```
'''text
order_items many ---> 1 customer_orders
'''
```

Code mapping:

```
'''java
@ManyToOne(fetch = FetchType.LAZY)
@JoinColumn(name = "order_id")
private CustomerOrder order;
'''
```

Sample seeded rows:

item_id	order_id	name	price
ITEM-1	ORD-1001	Shoes	2000
ITEM-2	ORD-1001	Shirt	1000
ITEM-3	ORD-1001	Watch	5000

3.3 TICKETS

Entity:

```
'''text
Ticket
'''
```

Java file:

```
'''text
src/main/java/com/personal/ai_chatbot/entity/Ticket.java
'''
```

Purpose:

Stores one complaint/request raised by the user.

Table:

```
'''text
tickets
'''
```

Columns:

Column	Java Type	DB Meaning	Notes
ticket_id	Long	Primary key	Auto-generated
user_id	String	User who raised the complaint	Used for recovery and duplicate checks
order_id	String	Order under complaint	References the order logically
issue_type	IssueType enum	Complaint issue type	DAMAGED, NOT_DELIVERED, WRONG_ITEM, OTHER
state	ConversationState enum	Current conversation state	START, ASK_ORDER_ID, SELECT_ITEM
status	TicketStatus enum	Business status	OPEN, RESOLVED, ESCALATED
refund_amount	BigDecimal	Final calculated refund	Set during final processing
created_at	Instant	Created timestamp	Set automatically before insert
updated_at	Instant	Updated timestamp	Updated automatically before update

Relationship:

```
'''text
tickets 1 ---> many ticket_items
'''
```

Code mapping:

```
'''java
@OneToMany(mappedBy = "ticket", cascade = CascadeType.ALL, orphanRemoval = true)
private List<TicketItem> items = new ArrayList<>();
'''
```

Important design point:

'tickets.order_id' is stored as a string instead of a JPA relation. This keeps the complaint domain simple and avoids accidental order updates through ticket logic.

Status meaning:

Status	Meaning
OPEN	Complaint is currently active
RESOLVED	Refund was initiated and ticket is complete
ESCALATED	Ticket needs human support

State meaning:

State	Meaning
START	New conversation
ASK_ORDER_ID	Bot is waiting for the order ID
SELECT_ITEMS	Bot is waiting for item selection
ASK_ISSUE	Bot is waiting for issue type
PROCESS_REFUND	Backend is ready to calculate refund and decide final status
RESOLVED	Ticket was resolved
ESCALATED	Ticket was escalated

3.4 TICKET_ITEMS

Entity:

```
'''text
TicketItem
'''
```

Java file:

```
'''text
src/main/java/com/personal/ai_chatbot/entity/TicketItem.java
'''
```

Purpose:

Stores the items selected for a complaint. These are copied from 'order_items' into 'ticket_items'.

Why copy the item data?

Because complaint history should remain stable. If the product name or price changes later in the order/product system, the ticket should still show what was selected at complaint time.

Table:

```
'''text
ticket_items
'''
```

Columns:

Column	Java Type	DB Meaning	Notes
id	Long	Primary key	Auto-generated
ticket_id	Long	Foreign key to tickets.ticket_id	Links selected item to complaint
item_id	String	Original order item ID	Example: ITEM-1
item_name	String	Copied product name	Example: Shoes

| price | BigDecimal | Copied price | Used for refund calculation |

Relationship:

```
'''text
ticket_items many ---> 1 tickets
'''
```

Code mapping:

```
'''java
@ManyToOne(fetch = FetchType.LAZY)
@JoinColumn(name = "ticket_id")
private Ticket ticket;
'''
```

3.5 CHAT_MESSAGES

Entity:

```
'''text
ChatMessage
'''
```

Java file:

```
'''text
src/main/java/com/personal/ai_chatbot/entity/ChatMessage.java
'''
```

Purpose:

Stores user and bot messages for audit/history and AI context.

Table:

```
'''text
chat_messages
'''
```

Columns:

Column	Java Type	DB Meaning	Notes
--- ---	---	---	---
id	Long	Primary key	Auto-generated
ticket_id	Long	Current ticket ID, if available	Can be null before ticket is created
user_id	String	User who sent/received message	Used to fetch recent context
sender	MessageSender enum	USER or BOT	Identifies who sent the message
message	String	Text content	Length 2000
timestamp	Instant	Message timestamp	Set automatically before insert

Important detail:

The first user message may not have a ticket yet. Example:

```
'''text
User: I have a refund issue
'''
```

At that moment, 'ticket_id' can be null because the ticket is only created after order validation.

4. REDIS SESSION STRUCTURE

DTO:

```
'''text
UserSession
'''
```

Java file:

```
'''text
src/main/java/com/personal/ai_chatbot/dto/UserSession.java
'''
```

Redis key:

```
'''text
session:{userId}
'''
```

Fields:

Field	Type	Meaning
userId	String	Current user
currentTicketId	Long	Active ticket ID
state	ConversationState	Current conversation step
selectedItemIds	List<String>	Selected item IDs
retryCount	int	Count of invalid/unclear attempts

Example Redis value:

```
'''json
{
  "userId": "user-1",
  "currentTicketId": 10,
  "state": "ASK_ISSUE",
  "selectedItemIds": ["ITEM-1", "ITEM-2"],
  "retryCount": 0
}
'''
```

Local fallback:

'RedisSessionService' also keeps an in-memory fallback map so local development still works if Redis is not running. PostgreSQL remains the permanent source of truth.

5. RELATIONSHIP DIAGRAM

```
'''text
customer_orders
  order_id PK
  user_id
  total_amount
  |
  | 1 to many
  v
order_items
Page 7
```

```

    item_id PK
    order_id FK
    name
    price

tickets
    ticket_id PK
    user_id
    order_id
    issue_type
    state
    status
    refund_amount
    created_at
    updated_at
    |
    | 1 to many
    v
ticket_items
    id PK
    ticket_id FK
    item_id
    item_name
    price

chat_messages
    id PK
    ticket_id nullable
    user_id
    sender
    message
    timestamp
'''

```

6. SERVICE RESPONSIBILITY MAP

```

| Layer/Service | Main Responsibility |
|---|---|
| ChatController | Receives WebSocket STOMP messages and returns bot response |
| ChatServiceImpl | Main state machine and orchestration |
| SessionService / RedisSessionService | Read/write active user session |
| TicketServiceImpl | Create/update tickets and selected ticket items |
| OrderServiceImpl | Validate order and parse item selection |
| RefundServiceImpl | Calculate deterministic refund from selected ticket items |
| DuplicateComplaintServiceImpl | Find previous ticket with same user/order/items/issue |
| EscalationServiceImpl | Apply escalation rules |
| AIService / OpenAIService | Intent, extraction, normalization, escalation signal |
| RuleBasedAIService | Fallback if no API key or API is unavailable |

```

7. WEBSOCKET DATA FLOW

Client connects to:

```

'''text
/ws
'''

```


Client sends message to:

```
'''text
/app/chat.sendMessage
'''
```

Server broadcasts response to:

```
'''text
/topic/public
'''
```

Request DTO:

```
'''json
{
  "userId": "user-1",
  "message": "I have a refund issue"
}
'''
```

Response DTO:

```
'''json
{
  "userId": "user-1",
  "ticketId": 10,
  "state": "SELECT_ITEMS",
  "status": "OPEN",
  "message": "Your order contains...",
  "timestamp": "2026-04-26T09:00:00Z"
}
'''
```

8. END-TO-END LOGIC FLOW

8.1 START STATE

Input example:

```
'''text
I have a refund issue
'''
```

Flow:

1. 'ChatController.sendMessage()' receives WebSocket payload.
2. 'ChatServiceImpl.handleMessage()' loads session from Redis.
3. If no session exists, it creates one with state 'START'.
4. User message is saved in 'chat_messages'.
5. 'aiService.isComplaintIntent()' checks if the message is complaint-related.
6. If complaint intent is found:
 - Redis session state becomes 'ASK_ORDER_ID'.
 - Bot asks for order ID.
7. Bot message is saved in 'chat_messages'.
8. Session is saved back to Redis.

Database activity:

```
'''text
INSERT INTO chat_messages (... sender=USER ...)
INSERT INTO chat_messages (... sender=BOT ...)
'''
```

No ticket is created yet because order ID is not known.

8.2 ASK_ORDER_ID STATE

Input example:

```
'''text
my order is ORD-1001
'''
```

Flow:

1. 'aiService.extractOrderId()' extracts 'ORD-1001'.
2. 'OrderServiceImpl.findUserOrder(orderId, userId)' verifies:
 - order exists
 - order belongs to the current user
3. 'TicketServiceImpl.createTicket(userId, orderId)' creates a ticket:
 - status = OPEN
 - state = SELECT_ITEMS
4. Redis session stores:
 - currentTicketId
 - state = SELECT_ITEMS
5. 'OrderServiceImpl.buildItemSelectionText()' formats order items for the bot.

Database activity:

```
'''text
SELECT customer_orders WHERE order_id = ? AND user_id = ?
INSERT INTO tickets (... status=OPEN, state=SELECT_ITEMS ...)
INSERT INTO chat_messages (... sender=USER ...)
INSERT INTO chat_messages (... sender=BOT, ticket_id=newTicketId ...)
'''
```

Tables used:

Table	Purpose
customer_orders	Validate order ownership
order_items	Show selectable item list
tickets	Create complaint record
chat_messages	Save conversation

8.3 SELECT_ITEMS STATE

Input examples:

```
'''text
1
1,2
all
'''
```

Flow:

1. Backend loads current ticket by 'currentTicketId'.
2. Backend reloads the order using ticket order ID and user ID.
3. 'OrderServiceImpl.selectItems()' parses item selection:
 - 'all' returns all order items
 - '1,2' returns selected item positions
 - invalid input returns empty list
4. 'TicketServiceImpl.saveSelectedItems()' copies selected order items into 'ticket_items'.
5. Ticket state changes to 'ASK_ISSUE'.
6. Redis session stores:
 - selected item IDs
 - state = ASK_ISSUE

Database activity:

```
'''text
SELECT tickets WHERE ticket_id = ?
SELECT customer_orders WHERE order_id = ? AND user_id = ?
SELECT order_items WHERE order_id = ?
DELETE old ticket_items for ticket if any
INSERT INTO ticket_items (... copied item data ...)
UPDATE tickets SET state = ASK_ISSUE
INSERT INTO chat_messages ...
'''
```

Why ticket_items are copied:

The selected item data becomes part of the complaint record. Refund is calculated from 'ticket_items', not directly from 'order_items'.

8.4 ASK_ISSUE STATE

Input examples:

```
'''text
damaged
the product arrived broken
wrong item
not received
'''
```

Flow:

1. 'aiService.normalizeIssueType()' converts messy text to one enum:
 - DAMAGED
 - NOT_DELIVERED
 - WRONG_ITEM
 - OTHER
2. Ticket is updated with 'issue_type'.
3. Ticket state becomes 'PROCESS_REFUND'.
4. 'DuplicateComplaintServiceImpl.findDuplicate()' checks for existing ticket with:
 - same user ID
 - same order ID
 - same issue type
 - same selected item IDs
5. If duplicate exists, backend returns duplicate message.
6. If no duplicate exists, backend continues to refund processing.

Database activity:

```
'''text
UPDATE tickets SET issue_type = ?, state = PROCESS_REFUND
Page 11
```

```
SELECT tickets WHERE user_id = ? AND order_id = ? AND issue_type = ?
SELECT ticket_items for comparison
'''
```

Duplicate handling:

```
| Existing Status | Bot Message |
|---|---|
| OPEN | You already have an active request for this issue. |
| RESOLVED | This issue has already been resolved for your order. If you still need help, I can
| ESCALATED | This issue has already been escalated to our support team. |
```

8.5 PROCESS_REFUND STATE

Flow:

1. Current ticket is loaded.
2. Recent messages are loaded from 'chat_messages'.
3. 'AIService.analyzeEscalation()' returns advisory signal.
4. 'RefundServiceImpl.calculateRefund()' sums selected ticket item prices.
5. 'EscalationServiceImpl.shouldEscalate()' checks escalation rules.
6. Ticket becomes either:
 - RESOLVED
 - ESCALATED
7. Final ticket status and refund amount are saved in 'tickets'.

Refund calculation:

```
'''java
ticket.getItems().stream()
    .map(item -> item.getPrice() == null ? BigDecimal.ZERO : item.getPrice())
    .reduce(BigDecimal.ZERO, BigDecimal::add);
'''
```

Important rule:

AI never calculates refund. Refund is always deterministic backend logic.

Escalation rules:

```
'''text
Escalate if:
refundAmount > 5000
OR retryCount > 2
OR latest message contains human/agent/manager
OR AI recommends escalation with confidence >= 0.75
'''
```

Database activity:

```
'''text
SELECT chat_messages WHERE user_id = ? ORDER BY timestamp DESC LIMIT 5
UPDATE tickets SET status = RESOLVED/ESCALATED, state = RESOLVED/ESCALATED, refund_amount = ?
INSERT INTO chat_messages (... final bot response ...)
'''
```

9. HOW DATA MOVES BETWEEN TABLES

9.1 ORDER TO TICKET

When order ID is valid:

```
'''text
customer_orders.order_id
customer_orders.user_id
      |
      v
tickets.order_id
tickets.user_id
'''
```

The ticket stores order ID and user ID, but does not copy every order detail.

9.2 ORDER ITEMS TO TICKET ITEMS

When user selects items:

```
'''text
order_items.item_id  ---> ticket_items.item_id
order_items.name     ---> ticket_items.item_name
order_items.price    ---> ticket_items.price
'''
```

This is a snapshot copy.

Example:

User selects '1,2'.

Order items:

Position	item_id	name	price
1	ITEM-1	Shoes	2000
2	ITEM-2	Shirt	1000

Ticket items inserted:

ticket_id	item_id	item_name	price
10	ITEM-1	Shoes	2000
10	ITEM-2	Shirt	1000

Refund:

```
'''text
2000 + 1000 = 3000
'''
```

9.3 CHAT MESSAGES TO AI

When processing refund:

```
'''text
chat_messages
  latest 3-5 messages
Page 13
```

```

      |
      v
AIService
      |
      v
AIEscalationResult
'''

```

AI result:

```

'''json
{
  "sentiment": "ANGRY",
  "escalationRecommended": true,
  "confidence": 0.92,
  "reason": "User expresses repeated dissatisfaction"
}
'''

```

Backend then validates:

```

'''text
if escalationRecommended == true and confidence >= 0.75
then escalation condition is true
'''

```

AI output is not written directly to the database. It only influences backend decision logic.

10. AI INTEGRATION LOGIC

Service:

```

'''text
OpenAIService
'''

```

Fallback:

```

'''text
RuleBasedAIService
'''

```

AI tasks:

AI Feature	Used In State	Backend Use
Complaint intent detection	START	Decide whether to ask for order ID
Order ID extraction	ASK_ORDER_ID	Extract order ID from natural text
Issue normalization	ASK_ISSUE	Convert text to IssueType enum
Escalation detection	PROCESS_REFUND	Advisory signal for escalation

OpenAIService sends:

```

'''text
current state
latest user message
recent messages
'''

```

OpenAIService expects structured output:

```
'''json
{
  "intent": "ORDER_COMPLAINT",
  "intentConfidence": 0.9,
  "orderId": "ORD-1001",
  "issueType": "DAMAGED",
  "issueConfidence": 0.9,
  "sentiment": "ANGRY",
  "escalationRecommended": true,
  "escalationConfidence": 0.92,
  "reason": "User is frustrated"
}
'''
```

Fallback behavior:

If 'OPENAI_API_KEY' is not configured or API call fails, the rule-based service still handles:

- simple complaint keywords
- order ID pattern with digits
- damaged/broken/wrong/not delivered issue text
- basic angry/frustrated escalation

11. SESSION RECOVERY

If Redis session is missing:

1. 'ChatServiceImpl.loadOrRecoverSession()' asks Redis first.
2. If Redis has no session, it queries latest recoverable ticket.
3. Recoverable ticket statuses:
 - OPEN
 - ESCALATED
4. A new 'UserSession' is rebuilt from the ticket:
 - userId
 - currentTicketId
 - state
 - selectedItemIds
5. Session is stored again.

Repository method:

```
'''java
findFirstByUserIdAndStatusInOrderByUpdatedAtDesc(...)
'''
```

This lets the conversation continue even after Redis TTL expiry.

12. EXAMPLE COMPLETE FLOW WITH TABLES

STEP A: USER STARTS

Message:

```
'''text
I have a refund issue
'''
```

Redis:

```
'''text
session:user-1.state = ASK_ORDER_ID
'''
```

chat_messages:

```
| ticket_id | user_id | sender | message |
|---|---|---|---|
| null | user-1 | USER | I have a refund issue |
| null | user-1 | BOT | Please provide your order ID. |
```

STEP B: USER SENDS ORDER ID

Message:

```
'''text
ORD-1001
'''
```

customer_orders lookup:

```
'''text
order_id = ORD-1001
user_id = user-1
'''
```

tickets:

```
| ticket_id | user_id | order_id | state | status |
|---|---|---|---|---|
| 10 | user-1 | ORD-1001 | SELECT_ITEMS | OPEN |
```

Redis:

```
'''text
session:user-1.currentTicketId = 10
session:user-1.state = SELECT_ITEMS
'''
```

STEP C: USER SELECTS ITEMS

Message:

```
'''text
1,2
'''
```

ticket_items:

```
| ticket_id | item_id | item_name | price |
|---|---|---|---|
| 10 | ITEM-1 | Shoes | 2000 |
| 10 | ITEM-2 | Shirt | 1000 |
```

tickets:

```
| ticket_id | state | status |
```



```
|---|---|---|
| 10 | ASK_ISSUE | OPEN |
```

Redis:

```
'''text
selectedItemIds = ["ITEM-1", "ITEM-2"]
state = ASK_ISSUE
'''
```

STEP D: USER PROVIDES ISSUE

Message:

```
'''text
damaged
'''
```

tickets:

```
| ticket_id | issue_type | state |
|---|---|---|
| 10 | DAMAGED | PROCESS_REFUND |
```

Duplicate check:

```
'''text
same user_id = user-1
same order_id = ORD-1001
same issue_type = DAMAGED
same item IDs = ITEM-1, ITEM-2
exclude current ticket_id = 10
'''
```

STEP E: REFUND DECISION

Refund:

```
'''text
ITEM-1 price 2000
ITEM-2 price 1000
refundAmount = 3000
'''
```

Escalation check:

```
'''text
3000 > 5000 ? false
retryCount > 2 ? false
message asks for human/agent/manager ? false
AI escalation confidence >= 0.75 ? depends on AI signal
'''
```

Final ticket:

```
| ticket_id | status | state | refund_amount |
|---|---|---|---|
| 10 | RESOLVED | RESOLVED | 3000 |
```

Bot:

```
'''text
Your refund of Rs.3000 has been initiated.
'''
```

13. TRANSACTION HANDLING

'ChatServiceImpl.handleMessage()' is annotated with:

```
'''java
@Transactional
'''
```

Meaning:

- The DB operations inside one chat message are part of one transaction.
- If a runtime error happens during ticket/item/message persistence, the transaction can roll back.
- Redis session write happens at the end, after response creation.

Note:

Redis is not part of the relational DB transaction. This is acceptable because PostgreSQL is the permanent truth and Redis can be recovered from tickets.

14. IMPORTANT BUSINESS RULES

1. AI must not calculate refund.
2. AI must not directly update ticket status.
3. AI must not write to the database.
4. Backend validates order ownership before ticket creation.
5. Backend stores selected complaint items in 'ticket_items'.
6. Duplicate complaint detection happens before final resolution.
7. High refund amount escalates.
8. Angry/frustrated users can be escalated through AI signal.
9. Out-of-scope messages are rejected.
10. Redis session expiry should not lose active tickets.

15. SUGGESTED FUTURE IMPROVEMENTS

1. Add explicit foreign key from 'tickets.order_id' to 'customer_orders.order_id'.
2. Add indexes:
 - 'tickets(user_id, status, updated_at)'
 - 'tickets(user_id, order_id, issue_type)'
 - 'chat_messages(user_id, timestamp)'
 - 'ticket_items(ticket_id)'
3. Store AI signal audit in a separate table if needed:
 - 'ai_analysis_logs'
4. Add a REST admin endpoint for loading orders.
5. Add integration tests for duplicate complaint and escalation flows.
6. Add a support-agent assignment table for escalated tickets.